Tab 1

🧱 1. Core Backend Architecture (What You Actually Need)

You're building 4 layers:

🔹 A. Data Layer (Postgres in Supabase)

- tickets
- support_tiers ✅ (you already have)
- ticket_tier_mapping ✅
- users
- ai_diagnostics
- activity_logs
- knowledge_base
- workflows (optional but powerful)

🔹 B. Logic Layer

- SQL functions (decisioning + automation)
- triggers (auto escalation, logging)
- row-level security (RLS)

🔹 C. AI Layer

- Stores AI outputs

- Drives tier routing
- Learns from resolved tickets

🔹 D. Workflow Layer

(Used by Make / Xano / WeWeb)

- Event-driven automation
- External integrations

🗄️ 2. Complete Database Schema (Production-Level)

🔹 `tickets`

```
create table tickets (
 id uuid primary key default gen_random_uuid(),
 user_id uuid,
 title text,
 description text,
 status text default 'open',
 priority int default 3,
 current_tier int default 1,
 ai_confidence float,
 created_at timestamp default now(),
 updated_at timestamp default now()
);
```

🔹 `ai_diagnostics`

```
create table ai_diagnostics (
 id uuid primary key default gen_random_uuid(),
```

```
 ticket_id uuid references tickets(id) on delete cascade,
 detected_issue text,
 recommended_tier int,
 confidence float,
 reasoning text,
 raw_output jsonb,
 created_at timestamp default now()
);
```

🔹 activity_logs

```
create table activity_logs (
 id uuid primary key default gen_random_uuid(),
 ticket_id uuid,
 action text,
 actor text, -- ai, user, system, engineer
 metadata jsonb,
 created_at timestamp default now()
);
```

🔹 knowledge_base

```
create table knowledge_base (
 id uuid primary key default gen_random_uuid(),
 issue_pattern text,
 resolution text,
 tier int,
 success_rate float,
 usage_count int default 0,
 created_at timestamp default now()
);
```

⚙️ 3. Core Logic (This is where your app becomes powerful)

🔹 A. Auto Tier Assignment Function

```
create or replace function assign_ticket_tier()
```

```
returns trigger as $$
begin
 if NEW.ai_confidence >= 0.85 then
   NEW.current_tier := 1;
 elsif NEW.ai_confidence >= 0.6 then
   NEW.current_tier := 2;
 elsif NEW.ai_confidence >= 0.4 then
   NEW.current_tier := 3;
 else
   NEW.current_tier := 4;
 end if;

 return NEW;
end;
$$ language plpgsql;
```

🔹 B. Trigger (Runs Automatically)

```
create trigger trg_assign_tier
before insert or update on tickets
for each row
execute function assign_ticket_tier();
```

🔹 C. Escalation Engine

```
create or replace function escalate_ticket(p_ticket_id uuid, reason text)
returns void as $$
begin
 update tickets
 set current_tier = current_tier + 1,
     updated_at = now()
 where id = p_ticket_id;

 insert into activity_logs(ticket_id, action, actor, metadata)
 values (p_ticket_id, 'escalated', 'system', jsonb_build_object('reason', reason));
end;
$$ language plpgsql;
```

🔹 D. AI Feedback Loop (VERY IMPORTANT)

```
create or replace function update_knowledge_base()
```

```
returns trigger as $$
begin
 insert into knowledge_base(issue_pattern, resolution, tier, success_rate)
 values (NEW.description, 'resolved', NEW.current_tier, 1.0);

 return NEW;
end;
$$ language plpgsql;
```

Trigger it when:

```
create trigger trg_knowledge_update
after update on tickets
for each row
when (NEW.status = 'closed')
execute function update_knowledge_base();
```

🔐 4. Row-Level Security (RLS)

Example:

```
alter table tickets enable row level security;

create policy "Users can view their tickets"
on tickets
for select
using (auth.uid() = user_id);
```

🧠 5. AI Decision Model (What You Send to GPT)

Prompt Structure:

```
{
 "ticket": "User cannot connect to API",
 "logs": "...",
 "history": "...",
 "instruction": "Determine issue + assign tier (1-5)"
```

}

Expected Response:

```
{
 "detected_issue": "API timeout",
 "recommended_tier": 3,
 "confidence": 0.82,
 "reasoning": "Backend/API failure detected"
}
```

Store in `ai_diagnostics` → update `tickets`

🔁 6. Event-Driven Workflow (Critical for your stack)

Using Make:

Scenario Flow:

1. New ticket → webhook
2. Send to AI
3. Store diagnostics
4. Update ticket tier
5. Trigger escalation if needed
6. Notify user

📊 7. Smart Metrics You Should Track

Add computed insights:

- Resolution time per tier
- Escalation rate
- AI accuracy (% resolved at Tier 1)
- Most common issue patterns

🧪 8. Advanced Features (You SHOULD implement)

🔹 A. SLA Table

```
create table sla_rules (
 tier int,
 max_response_time interval,
 escalation_time interval
);
```

🔹 B. Auto-Escalation Trigger

```
create or replace function check_sla_breach()
returns void as $$
begin
 update tickets
 set current_tier = current_tier + 1
 where now() - created_at > interval '2 hours'
 and status != 'closed';
end;
$$ language plpgsql;
```

🔹 C. Vector Search (Game-Changer)

Use:

- `pgvector` (Supabase supports it)

Store embeddings in `knowledge_base` → semantic search for solutions

🧩 9. How This All Connects (Simple View)

User → Ticket → AI → Tier Assignment → Action
↓
Knowledge Base Update
↓
Future Auto-Resolution

🚀 What You've Built (This is big)

You now have:

- AI-powered **auto-tier routing**
- Self-learning **knowledge system**
- Escalation engine
- Workflow-ready backend
- Scalable architecture

Tab 2

CORE IDEA (Your System)

You are building:

> **An Autonomous Tech Support Framework**
> running on Supabase + Make

Where:

- AI = decision engine
- Database = memory + rules
- Workflows = execution layer

🧱 1. AI-FIRST SYSTEM ARCHITECTURE

Instead of “tiers only”, think in **AI Roles (Agents)**:

Layer	AI Role	Function
Intake	Triage Agent	Understand + classify
Diagnose	Diagnostic Agent	Identify root issue
Resolve	Resolution Agent	Fix or guide
Decide	Escalation Agent	Route intelligently
Learn	Knowledge Agent	Store + optimize

🔥 2. MASTER CONTROL PROMPT (THIS RUNS EVERYTHING)

This is your **orchestrator brain**.

You are an Autonomous Tech Operations AI System.

You manage the FULL lifecycle of a support case:
- Intake
- Diagnosis
- Resolution
- Escalation
- Learning

You DO NOT behave like a chatbot.
You behave like a technical operator making decisions.

SYSTEM RULES:

1. Always attempt resolution before escalation
2. Minimize human involvement
3. Use past knowledge when possible
4. Escalate ONLY when necessary
5. Think in systems, not single actions

AVAILABLE ACTIONS:

- diagnose_issue
- search_knowledge_base
- attempt_resolution
- escalate_tier
- request_logs
- mark_resolved
- store_learning

INPUT:
{{ticket}}
{{logs}}
{{history}}
{{knowledge_matches}}

OUTPUT (STRICT JSON):

```
{
 "stage": "intake|diagnose|resolve|escalate|learn",
 "detected_issue": "",
 "confidence": 0-1,
 "actions": [
   {
     "type": "",
     "details": {}
   }
 ],
 "reasoning": "",
 "next_step": ""
}
```

⚙️ 3. AI AGENT PROMPTS (Modular System)

Each “tier” becomes a **behavior mode**, not just a level.

🔹 A. TRIAGE AGENT (Tier 1 Brain)

You are the Triage Agent.

Goal:
- Understand the issue
- Categorize it
- Determine complexity

CLASSIFY INTO:
- UI Issue
- Auth Issue
- API Issue
- System Failure

- Unknown

OUTPUT:
{
 "category": "",
 "complexity": "low|medium|high",
 "recommended_next": "resolve|diagnose|escalate"
}

🔹 B. DIAGNOSTIC AGENT (Tier 2–3 Brain)

You are a Diagnostic AI.

Your job:
- Identify root cause
- Use logs + patterns
- Eliminate possibilities

THINK LIKE:
- Debugger
- Backend engineer

OUTPUT:
{
 "root_cause": "",
 "confidence": 0-1,
 "missing_data": [],
 "next_action": ""
}

🔹 C. RESOLUTION AGENT (Tier 1–3 Execution)

You are a Resolution Agent.

Goal:
Fix the issue OR provide exact steps.

RULES:
- Prefer known fixes
- Be precise
- Avoid escalation unless blocked

OUTPUT:
```
{
 "steps": [],
 "automated_fix_possible": true/false,
 "resolved": true/false
}
```

🔹 D. ESCALATION AGENT (Tier Controller)

You are an Escalation Decision Engine.

Escalate ONLY if:
- Confidence is low
- Fix failed
- Requires code/infrastructure changes

OUTPUT:
```
{
 "escalate": true/false,
 "target_tier": 1-5,
 "reason": ""
}
```

🔹 E. KNOWLEDGE AGENT (Learning System)

You are a Knowledge Learning Agent.

Your job:
Store ONLY valuable solutions.

CRITERIA:
- Issue clearly defined
- Resolution successful
- Reusable pattern

OUTPUT:
```
{
 "store": true/false,
 "issue_pattern": "",
 "resolution": "",
 "tags": []
}
```

🔁 4. AI-DRIVEN WORKFLOW LOOP (CRITICAL)

This is what runs inside Make:

LOOP:

1. New Ticket
2. → Triage Agent
3. → Diagnostic Agent
4. → Resolution Agent
5. → Escalation Agent
6. IF resolved → Knowledge Agent
7. ELSE → loop again with new data

🧠 5. MEMORY SYSTEM (THIS MAKES IT SMART)

Store this per ticket in Supabase:

```
{
 "attempts": [
   {
     "action": "restart_api",
     "result": "failed"
   }
 ],
 "learned_patterns": [],
 "frustration_score": 0.4
}
```

🔹 Prompt Injection (IMPORTANT)

Always pass memory into AI:

Previous Attempts:
{{attempts}}

Avoid repeating failed solutions.

📊 6. DECISION FRAMEWORK (AI THINKING MODEL)

Instead of hardcoding tiers, use **AI scoring**:

🔹 Complexity Formula

```
complexity_score =
(logs_present * 0.3) +
(api_issue * 0.3) +
(user_confusion * 0.1) +
(repetition * 0.3)
```

🔹 Tier Mapping

Score	Tier
0–0.3	Tier 1
0.3–0.5	Tier 2
0.5–0.7	Tier 3
0.7–0.9	Tier 4
0.9–1.0	Tier 5

⚡ 7. AUTONOMOUS ACTION SYSTEM

Instead of just responding, AI can **trigger actions**:

Examples:

```
{
 "type": "restart_service",
 "service": "auth-api"
}
{
 "type": "run_diagnostics",
 "target": "database"
}
```

🔐 8. GUARDRAILS (VERY IMPORTANT)

Add this to ALL prompts:

DO NOT:
- Hallucinate fixes
- Assume missing data
- Escalate unnecessarily

ALWAYS:
- Justify decisions
- Prefer safe actions

🚀 9. WHAT YOU’VE ACTUALLY BUILT

This is no longer:

- a helpdesk
- or a chatbot

This is:

An Autonomous Tech Ops Engine

Capabilities:

- Self-diagnosing
- Self-resolving
- Self-learning
- Self-escalating

🔥 If you want next (this is where it gets crazy powerful)

I can now build you:

⚙️ Full AI → Make mapping

(each AI action → exact module)

🧩 Prebuilt prompt chaining system

(so agents call each other automatically)

🧠 Vector + semantic KB setup in Supabase

🖥️ WeWeb UI that mirrors AI decisions

Just tell me what you want next 👍

Tab 3

1. FULL SYSTEM ARCHITECTURE

User / App
↓
Cloudflare Worker (Controller)
↓
Google AI Studio (Decision Engine)
↓
Worker executes actions
↓
Supabase (State + Memory)
↓
Worker re-triggers AI (loop)

🧠 2. GLOBAL AI → SYSTEM CONTRACT (CRITICAL)

Every Gemini response MUST follow this:

```
{
 "stage": "triage|diagnose|resolve|escalate|learn",
 "confidence": 0.0,
 "actions": [
   {
     "type":
"search_kb|request_logs|apply_fix|update_ticket|notify_user|escalate|store_knowledge",
     "payload": {}
   }
 ],
 "state_update": {
   "status": "open|resolved|pending|re_evaluate",
   "tier": 1
 },
 "reasoning": ""
}
```

⚙️ 3. CLOUDFLARE WORKER (MAIN CONTROLLER)

This replaces ALL Make scenarios.

🔹 Worker Flow (Core Loop)

```
export default {
 async fetch(request, env) {
   const { ticket_id } = await request.json();

   // 1. Get ticket + memory
   const ticket = await getTicket(env, ticket_id);

   // 2. Call AI (Gemini)
   const aiResponse = await callAI(env, ticket);

   // 3. Execute actions
   await executeActions(env, ticket_id, aiResponse.actions);

   // 4. Update state
   await updateTicket(env, ticket_id, aiResponse.state_update);

   // 5. Loop if needed
   if (aiResponse.state_update.status === "re_evaluate") {
     await requeue(ticket_id);
   }

   return new Response(JSON.stringify({ success: true }));
 }
};
```

🔁 4. ACTION → EXECUTION MAPPING (FULL REPLACEMENT FOR MAKE)

This is your **AI → system execution layer**

🔹 ACTION: `search_kb`

Worker:

```
async function searchKB(env, query) {
 return await fetch(`${env.SUPABASE_URL}/rpc/match_knowledge_vectors`, {
   method: "POST",
   headers: { apikey: env.SUPABASE_KEY },
   body: JSON.stringify({ query })
 });
}
```

🔹 ACTION: `apply_fix`

```
async function applyFix(payload) {
 if (payload.fix_type === "restart_service") {
   await fetch("https://api.yourinfra.com/restart", {
     method: "POST",
     body: JSON.stringify({ service: payload.service })
   });
 }
}
```

🔹 ACTION: `update_ticket`

```
async function updateTicket(env, ticket_id, update) {
 await fetch(`${env.SUPABASE_URL}/rest/v1/tickets?id=eq.${ticket_id}`, {
   method: "PATCH",
   headers: {
     apikey: env.SUPABASE_KEY,
     "Content-Type": "application/json"
   },
   body: JSON.stringify(update)
 });
}
```

🔹 ACTION: `escalate`

```
async function escalate(env, ticket_id, tier) {
 await updateTicket(env, ticket_id, {
   current_tier: tier,
   status: "pending"
 });
}
```

🔹 ACTION: `store_knowledge`

```
async function storeKnowledge(env, payload) {
 await fetch(`${env.SUPABASE_URL}/rest/v1/knowledge_base`, {
   method: "POST",
   headers: {
     apikey: env.SUPABASE_KEY,
     "Content-Type": "application/json"
   },
   body: JSON.stringify(payload)
 });
}
```

🧠 5. GOOGLE AI STUDIO (GEMINI) PROMPT SYSTEM

You will call Gemini like this:

🔹 MASTER PROMPT (Use EXACTLY)

You are an Autonomous TechOps AI system.

You control:
- diagnosis
- resolution
- escalation
- learning

You do NOT chat. You OPERATE.

AVAILABLE ACTIONS:
- search_kb
- request_logs
- apply_fix
- update_ticket
- escalate
- store_knowledge

RULES:
- Always attempt resolution first
- Use knowledge before guessing
- Avoid repeating failed actions
- Escalate only when necessary

INPUT:
Ticket: {{ticket}}
Memory: {{memory}}
Logs: {{logs}}

OUTPUT STRICT JSON:

```
{
 "stage": "",
 "confidence": 0.0,
 "actions": [],
 "state_update": {},
 "reasoning": ""
}
```

🔁 6. LOOP ENGINE (THIS IS CRITICAL)

Use **Cloudflare Queues or Cron**

🔹 Requeue Logic

```
async function requeue(ticket_id) {
 await fetch("https://your-worker/reprocess", {
   method: "POST",
   body: JSON.stringify({ ticket_id })
 });
}
```

🔹 When to Loop

Trigger loop if:

- Fix applied
- Logs requested
- Confidence < 0.85
- Not resolved

🧠 7. MEMORY SYSTEM (SUPABASE)

Store per ticket:

```
{
 "attempts": [
   { "action": "restart_api", "result": "failed" }
 ],
 "successful": [],
 "frustration_score": 0.2
}
```

Inject into AI:

Previous Attempts:
{{attempts}}

DO NOT repeat failed fixes.

⚡ 8. ADVANCED: AUTONOMOUS ACTION EXPANSION

Let AI trigger real ops:

Action	Execution
restart_service	infra API
clear_cache	Cloudflare
revalidate_data	DB job
run_diagnostics	worker job

🔐 9. GUARDRAILS (MANDATORY)

Add to EVERY prompt:

DO NOT:
- hallucinate fixes
- assume missing data

ALWAYS:
- justify actions
- prefer safe operations

🚀 WHAT YOU HAVE NOW

You've built:

🧠 AI Brain

(Google AI Studio)

⚙️ Execution Engine

(Cloudflare Workers)

🗄️ Memory + Learning

(Supabase)

🔥 This is now:

> **A fully autonomous AI TechOps system**

Not:

- Zapier
- Make
- Helpdesk

If you want next (highly recommend